

Add Two Numbers — Linked List

English Study Notebook • Based on the supplied video transcript

1. Introduction

The problem is about adding two numbers represented by singly linked lists. The transcript focuses on the important observation that the digits are stored in reverse order, so the first node represents the least significant digit. This lets us perform addition directly from the head of each list, just like ordinary column addition from right to left.

2. Problem Statement

You are given two non-empty singly linked lists representing two non-negative integers. Each node contains one digit, and the digits are stored in reverse order. Add the two numbers and return the sum as a linked list in the same reverse-order format.

Example: [5 → 4] represents 45, while [4] represents 4. Their sum is 49, so the result is [9 → 4].

3. Small Concepts Required

Reverse-order storage: The head contains the ones digit, the next node contains the tens digit, and so on.

Carry: For each position, calculate $\text{sum} = \text{digit1} + \text{digit2} + \text{carry}$. The result digit is $\text{sum} \% 10$ and the new carry is $\text{sum} / 10$.

Missing digit: If one list ends earlier, treat its missing digit as 0.

Dummy node: A temporary starting node makes it easy to attach result nodes. The actual answer begins at `dummy.next`.

4. Approach Explained

Approach — Direct Linked-List Addition with Carry

Because the digits are already in reverse order, no reversing of the lists is required. Keep one pointer on each input list and a carry value. At every position, read the available digits, add the carry, create one result node, and move the input pointers forward. Continue while either list still has a node or a carry remains.

Important loop condition: `l1 != null || l2 != null || carry != 0`. The carry condition is necessary for cases such as $9 + 1 = 10$, where both input lists can finish but one final result node still has to be created.

5. Pseudocode

```
create dummy node
temp = dummy
carry = 0

while l1 is not empty OR l2 is not empty OR carry is not 0:
    sum = 0

    if l1 exists:
        add l1.data to sum
        move l1 to next node

    if l2 exists:
        add l2.data to sum
        move l2 to next node

    add carry to sum

    carry = sum / 10
    digit = sum % 10
```

```

create a node containing digit
attach it after temp
move temp to the new node

return dummy.next

```

6. Complete Java Code

```

class Solution {

    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {

        // Dummy node helps us build the result list
        ListNode dummy = new ListNode();

        // temp always points to the last node of the result
        ListNode temp = dummy;

        // Carry from the previous digit addition
        int carry = 0;

        // Continue while either list has nodes
        // or a carry is still remaining
        while (l1 != null || l2 != null || carry != 0) {

            int sum = 0;

            // Take digit from the first list
            if (l1 != null) {
                sum += l1.data;
                l1 = l1.next;
            }

            // Take digit from the second list
            if (l2 != null) {
                sum += l2.data;
                l2 = l2.next;
            }

            // Add carry from the previous position
            sum += carry;

            // Calculate the new carry
            carry = sum / 10;

            // Calculate the digit stored in this result node
            ListNode node = new ListNode(sum % 10);

            // Attach the node to the result
            temp.next = node;

            // Move temp to the newly created node
            temp = temp.next;
        }

        // Skip the dummy node
        return dummy.next;
    }
}

```

7. Visual Representation

Input lists:

```

l1: 5 → 4 → NULL      (45)
l2: 4 → NULL           (4)

```

Because the lists store digits in reverse order, addition starts at the heads:

```

5 → 4 → NULL
+
4 → NULL

```

9 → 4 → NULL

Result: dummy $\rightarrow$ 9 $\rightarrow$ 4 $\rightarrow$ NULL

actual answer starts at dummy.next

Pointer / data relationship

Component	Purpose
l1	Reads digits from the first number
l2	Reads digits from the second number
carry	Stores the value passed to the next digit
temp	Points to the last node in the result list
dummy.next	Points to the first actual result node

Carry Example

9 → 9 → NULL

1 → NULL

$9 + 1 = 10 \rightarrow \text{digit } 0, \text{ carry } 1$

$$9 + 0 + 1 = 10 \rightarrow \text{digit } 0, \text{ carry } 1$$
$$0 + 0 + 1 = 1 \rightarrow \text{digit } 1, \text{ carry } 0$$

Result: 0 → 0 → 1 → NULL

8. Important Code Lines

while (l1 != null || l2 != null || carry != 0) — keeps processing until both input lists are finished and no carry remains.

carry = sum / 10 — extracts the carry for the next position.

new ListNode(sum % 10) — extracts the digit that belongs in the current result position.

temp.next = node; temp = temp.next; — attaches the new node and moves the result pointer forward.

return dummy.next; — returns the actual result list, excluding the helper dummy node.

9. Edge Cases

Case	What happens
Both lists have the same length	Add corresponding digits normally.
One list is shorter	Missing digit is treated as 0.
Carry occurs	Carry is added to the next position.
Final carry remains	The loop creates one additional result node.

10. Time and Space Complexity

Time: $O(\max(n, m))$ — each node from the two input lists is processed once.

Auxiliary space: $O(1)$ — only a constant number of pointers and the carry variable are used, excluding the result list.

Output space: $O(\max(n, m))$ — the result list may contain one node for every processed digit plus a possible final carry node.

11. Conclusion

The key idea is that reverse-order storage already puts the least significant digits at the heads of the lists. Therefore, the problem can be solved in one traversal using two pointers and a carry. The dummy node simplifies result construction, and `dummy.next` gives the actual head of the answer.